

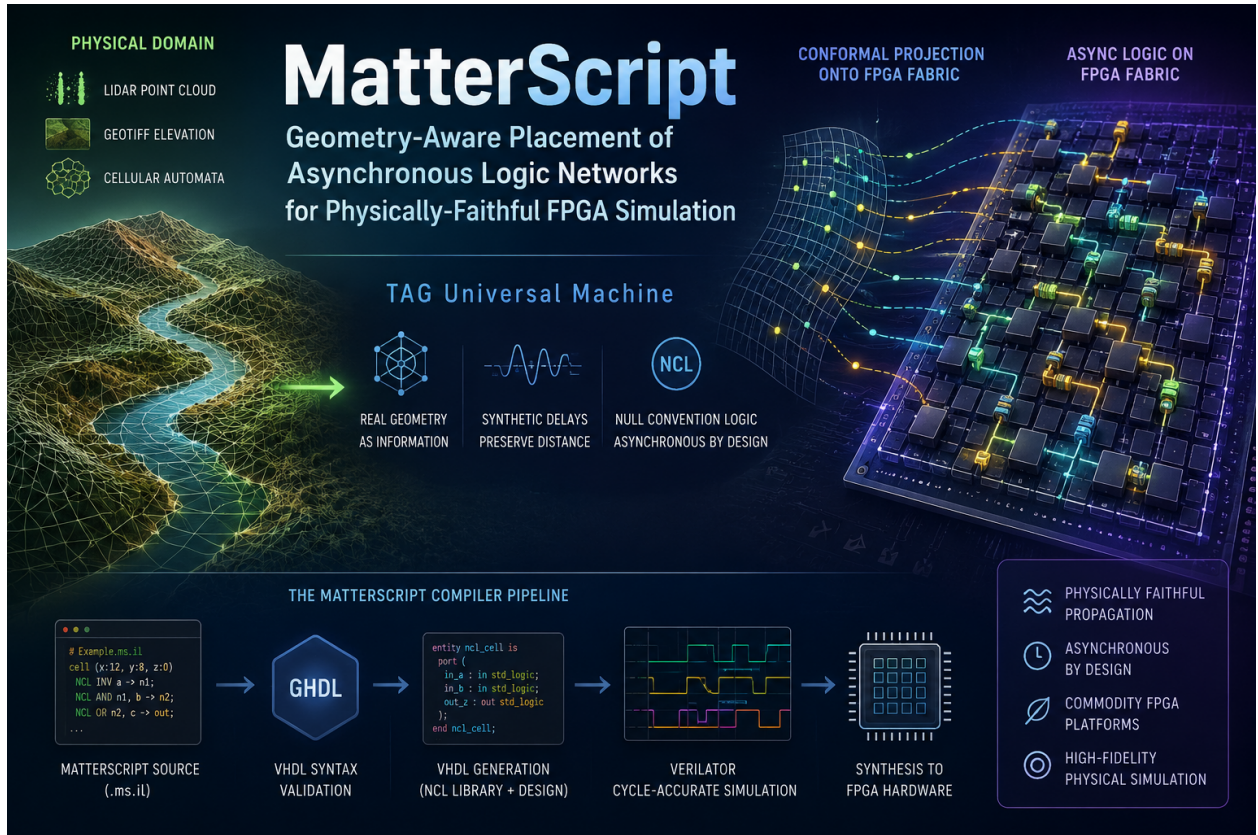
MatterScript: Geometry-Aware Placement of Asynchronous Logic Networks for Physically-Faithful FPGA Simulation

TAG Universal Machine

July 16, 2026

Abstract

We present MatterScript, a domain-specific language and compiler toolchain for geometry-aware placement of asynchronous logic networks on commodity FPGA hardware. MatterScript introduces the principle that the physical domain being simulated should determine the spatial arrangement of logic elements on the FPGA fabric, with synthetic propagation delays encoding geometric distances that cannot be preserved exactly by the projection. This produces simulations whose propagation behavior mirrors the physical domain without requiring custom hardware substrates. The system is grounded in Karl Fant’s Null Convention Logic and Invocation Language, which make propagation delay a first-class computational parameter rather than a side effect to be minimized. The compiler pipeline proceeds from `.ms.il` source through GHDL syntax validation to Verilator cycle-accurate simulation, and is designed to extend to standard FPGA synthesis tools. MatterScript additionally provides cellular automata as generative primitives for synthetic domain geometry, and finite state machine descriptions as a convenience layer over the full Invocation Language model.



Contents

1	Introduction	4
2	Background	4
2.1	The Limitations of Clock-Driven FPGA Simulation	4
2.2	Null Convention Logic	5
2.3	Karl Fant's Invocation Language	5
2.4	Conformal Mapping	6
3	The MatterScript Language	6
3.1	Design Philosophy	6
3.2	The FSM Dialect	6
3.3	The Invocation Language Dialect	7
3.3.1	Signal Encoding	7
3.3.2	Name Composition and ROM Synthesis	7
3.4	The Geo Dialect	8
4	Geometry-Aware Placement	8
4.1	The Two-Phase Pipeline	8
4.2	Why Real Geometry Beats Synthetic Approximation	9

4.3	Conformal Projection onto the FPGA Fabric	9
4.4	Synthetic Delay Elements in NCL	10
4.5	Cell Reference Syntax	10
5	Physical Simulation on the Geometry-Aware Fabric	11
5.1	Lattice Boltzmann Fluid Dynamics	11
5.2	The Digital Wind Tunnel	11
5.3	Comparison with Conventional FPGA Simulation	12
6	The Compiler Pipeline	12
6.1	Overview	12
6.2	The IL Parser	12
6.3	VHDL Generation	13
6.4	Build System	13
7	Implementation Status	13
8	Related Work	14
9	Conclusion	15

1 Introduction

Conventional FPGA-based simulation treats the logic fabric as a generic computing substrate. Place-and-route tools optimize for minimum wire length and maximum clock frequency, packing logic as densely as possible without regard for any geometric meaning the circuit might carry. A fluid dynamics simulation of a riverbed occupies the same undifferentiated grid as any other computation of similar size.

This paper argues that for physical simulation workloads, this is the wrong optimization objective. When simulating a physical domain — a fluid system, a structural load, an electromagnetic field — the geometry of the domain *is* information. Cells that are adjacent in the physical domain should be adjacent on the logic fabric. Cells that are far apart in the physical domain should have proportionally longer signal paths between them. The propagation of information through the simulation should mirror the propagation of effects through the physical system.

MatterScript makes this concrete through three connected ideas:

1. **Geometry-aware placement:** physical domain geometry, sourced from LIDAR point clouds, GeoTIFF elevation data, or cellular automata generation, is projected onto the FPGA fabric using conformal mapping. Logic cells are placed at projected positions.
2. **Delay synthesis:** distances distorted by the projection are compensated with synthetic delay elements, so that propagation timing across the fabric matches geodesic distances in the original domain.
3. **Null Convention Logic:** Karl Fant’s asynchronous logic model [1] makes propagation delay a computational parameter rather than a clock-constrained nuisance. NCL circuits fire when their inputs are complete, not when a clock edge arrives, making delay synthesis a natural and meaningful operation.

The result is a simulation that runs on a cheap commodity FPGA but whose propagation behavior is faithful to the physical geometry of the domain being modeled — analogous to a wind tunnel, but implemented in reconfigurable digital logic.

2 Background

2.1 The Limitations of Clock-Driven FPGA Simulation

Synchronous FPGA designs coordinate computation with a global clock signal. Every register updates on the same clock edge, regardless of whether the data feeding it has meaningfully changed. For physical simulations this creates several problems:

- **Artificial synchronization:** physical phenomena do not propagate synchronously. A pressure wave in a fluid does not update every cell simultaneously; it travels at a finite speed determined by the medium. Clocked simulation imposes a uniform time step that obscures this.

- **Geometry-blind placement:** standard place-and-route has no mechanism to express that two cells should be placed far apart because they represent physically distant locations. Distance in the domain is not represented in the circuit.
- **Staircase boundary approximation:** mapping irregular physical domains onto a regular grid introduces staircase artifacts at boundaries that affect simulation accuracy.

2.2 Null Convention Logic

Null Convention Logic [1] is an asynchronous logic paradigm that eliminates the global clock. Every signal carries a *completeness* indicator in addition to its data value. A signal in the NULL state carries no data. A signal in the DATA state carries a valid value. Computation proceeds when all inputs to a function transition from NULL to DATA, not on a clock edge.

For geometry-aware simulation, NCL has a decisive advantage: propagation delay is not noise to be minimized but a property of the network that can be deliberately designed. A synthetic delay element inserted between two logic cells is not a timing violation — it is a faithful representation of the physical distance between the locations those cells represent.

Additional properties of NCL relevant to physical simulation:

- **Natural pipeline correctness:** a cell cannot update until all its neighbor inputs are complete, preventing the data hazards that require careful pipeline staging in synchronous designs.
- **Network transparency:** the completeness handshake operates identically across on-chip connections, board-level buses, and network links, enabling seamless distribution of large simulations.
- **Sequentiality as emergence:** sequential behavior arises from data dependency chains rather than imposed clock ordering, correctly modeling the causal structure of physical propagation.

2.3 Karl Fant’s Invocation Language

Fant’s Invocation Language (IL) [1] provides a formal notation for NCL computation. The fundamental concept is the *theng*: something that has a location in the network and asserts a value. Thengs occupy *places*, which are named locations. Destination places (written `$name`) receive tokens from the outer context. Source places (written `name<>`) emit tokens once complete.

The IL’s most distinctive feature is *name composition*: destination place values are concatenated to form lookup keys into constant tables, implementing arbitrary combinational logic without conditional syntax. A composition `$a$b()` with `$a = 1` and `$b = 3` forms the key `13` and returns the corresponding table value. If no entry matches, the place remains NULL indefinitely — partial functions are expressed structurally rather than through error handling.

2.4 Conformal Mapping

A conformal map preserves angles locally while allowing area distortion. For the projection of a physical domain onto an FPGA fabric, conformal mapping minimizes the distortion of local geometry while fitting the domain into the available rectangular grid. MeshLab [3] implements several conformal UV parameterization algorithms suitable for this purpose. Distortion introduced by the mapping is measured per-cell and compensated by synthetic delay elements proportional to the distortion magnitude.

3 The MatterScript Language

3.1 Design Philosophy

MatterScript is organized around three dialects sharing a common flat, keyword-driven syntax:

- **.ms.fsm** — finite state machine descriptions, compiled to clocked VHDL as a convenience layer over the IL model
- **.ms.il** — Invocation Language networks, compiled to completeness-driven combinational VHDL
- **.ms.geo** — cellular automata substrate descriptions for synthetic domain geometry generation

A fourth dialect, **.ms.ca**, is under development for coupled physical simulation with geometry-aware placement.

3.2 The FSM Dialect

The FSM dialect provides concise notation for finite state machines, compiled to two-process VHDL (one clocked state register, one combinational next-state function). It is a convenience layer: in the full MatterScript model, finite state machines are IL networks in which exactly one token is in flight at any time, sequentiality being a degenerate case of concurrency.

Listing 1: CoffeeShop FSM

```
machine CoffeeShop

state Greeting
state TakeOrder
state Payment
state PrepareDrink
state Complete

event CustomerArrived
event OrderPlaced
```

```

event PaymentApproved
event DrinkReady

transition Greeting TakeOrder CustomerArrived
transition TakeOrder Payment OrderPlaced
transition Payment PrepareDrink PaymentApproved
transition PrepareDrink Complete DrinkReady

```

3.3 The Invocation Language Dialect

The IL dialect implements Fant’s formalism. Definitions are flat: all definitions live at the top level and reference each other by name, reflecting the underlying graph structure of the network.

Listing 2: 2-bit integer adder in IL

```

add[( $\$a$   $\$b$ )(result<>)
  result< $\$a\$b$ ()>
|
   $\$a\$b$ () : 00:0 01:1 02:2 03:3
           10:1 11:2 12:3 13:4
           20:2 21:3 22:4 23:5
           30:3 31:4 32:5 33:6
]

add((1 3)(x<>))

```

3.3.1 Signal Encoding

MatterScript IL compiles to VHDL using an 8-bit packed signal encoding:

$$\text{signal}[7:0] = \{\text{data}[6:0], \text{valid}\}$$

`signal[0]` is the valid (completeness) bit. `signal[7:1]` carries the 7-bit data payload, supporting up to 127 distinct token values. This encoding is uniform across all place kinds so connections between networks require no type conversion. Deasserting the valid bit freezes a token without destroying its value, implementing the NULL state while preserving DATA for later release.

3.3.2 Name Composition and ROM Synthesis

Name composition synthesizes to signal concatenation followed by a combinational case statement:

Listing 3: Generated VHDL for the add definition (excerpt)

```

-- key composition: concatenate 7-bit data fields
ab_key <= a(7 downto 1) & b(7 downto 1);

```

```

-- ROM lookup: fires only when complete
process(ab_key, complete) begin
  ab_data  <= (others => '0');
  ab_valid <= '0';
  if complete = '1' then
    case ab_key is
      when "00000010000011" => ab_data <= "100"; ab_valid <= '1';
      when others => null;
    end case;
  end if;
end process;

result <= ab_data & ab_valid;

```

Each character of the composition key encodes one destination's value as a 7-bit field. The key width scales with the number of contributing destinations. Unmatched keys leave the output NULL, correctly implementing partial functions.

3.4 The Geo Dialect

The geo dialect drives one-dimensional cellular automata to generate synthetic domain geometry:

Listing 4: Synthetic terrain via Rule 30

```

namespace terrain
seed entropy42
cald rule 30 width 128 steps 64
height_scale 2.4
export obj terrain.obj

```

The CA spacetime diagram is interpreted as a height map, producing a triangulated mesh exported as OBJ or STL. Rule 30 [2] generates non-periodic, statistically complex surfaces suitable for testing geometry-aware placement on irregular domains. For real-world applications, the geo dialect is bypassed and meshes are supplied directly from LIDAR or GeoTIFF sources via MeshLab preprocessing.

4 Geometry-Aware Placement

4.1 The Two-Phase Pipeline

MatterScript's geometry-aware placement pipeline has two phases separated by a well-defined interface: the mesh file.

Phase 1: Domain geometry acquisition and preprocessing. The physical domain geometry is acquired from measurement (LIDAR point clouds, GeoTIFF elevation data, photogrammetry) or generated synthetically (cellular automata). MeshLab [3] converts the

source data to a triangulated mesh, computes geodesic distances, performs UV parameterization, and annotates vertices with curvature and local area distortion. The output is a standard mesh file with per-vertex metadata.

Phase 2: Logic synthesis and geometry-aware placement. MatterScript reads the annotated mesh and maps the IL network onto the FPGA fabric. Each theng is assigned to a mesh vertex. The UV parameterization provides a 2D coordinate for each vertex, which is scaled to the FPGA fabric dimensions. Thengs are placed at these projected positions. Connections between thengs are routed along projected edge paths. Synthetic delay elements are inserted proportional to the distance distortion introduced by the projection.

4.2 Why Real Geometry Beats Synthetic Approximation

For real-world applications the domain geometry is known from measurement. Using actual LIDAR data of a riverbed rather than a CA approximation of it means:

- No approximation error in the domain shape
- Natural handling of irregular boundaries without staircase artifacts
- Direct correspondence between simulation cells and physical locations, enabling data assimilation from field sensors
- The simulation can be validated against physical measurements at known locations

The CA geo dialect remains valuable for synthetic benchmarks, for domains where measurement data is unavailable, and for exploring how different surface topologies affect simulation behavior.

4.3 Conformal Projection onto the FPGA Fabric

The FPGA fabric is a 2D rectangular array of logic elements. The physical domain surface is in general a curved 2D manifold embedded in 3D space. Projecting one onto the other requires a map from the surface to the rectangle.

MeshLab’s UV parameterization computes a conformal map that minimizes angular distortion. Cells that are adjacent on the surface map to nearby positions in the UV square. The UV coordinates are scaled to the FPGA fabric dimensions to produce integer placement coordinates for each theng.

Area distortion introduced by the conformal map is computed per-edge as:

$$\delta_{ij} = \left| \frac{d_{UV}(i, j)}{d_{geo}(i, j)} - 1 \right|$$

where d_{UV} is the Euclidean distance between projected positions and d_{geo} is the geodesic distance on the original surface. A synthetic delay of magnitude proportional to δ_{ij} is inserted on the connection between thengs i and j , compensating for the distortion.

4.4 Synthetic Delay Elements in NCL

In a synchronous design, synthetic delays are difficult to implement reliably — they fight against the clock and timing closure tools may remove them as optimization. In NCL, a delay element is simply a theng that introduces latency between its input becoming complete and its output becoming complete:

Listing 5: NCL delay element

```
-- d clock cycles of delay on an 8-bit NCL signal
process(clk) begin
  if rising_edge(clk) then
    delay_reg <= delay_reg(d-2 downto 0) & input;
  end if;
end process;
output <= delay_reg(d-1);
```

Because NCL logic does not require a global timing closure, delay elements of varying lengths can be freely mixed in the same design. The timing of each connection is determined by the physical distance it represents, not by a global constraint. This is the key enabling property that makes geometry-aware placement practical in NCL but not in synchronous logic.

4.5 Cell Reference Syntax

MatterScript extends the IL syntax with cell references that couple logic network thengs directly to simulation domain cells:

Listing 6: Cell coupling syntax

```
// observer: reads cell state without affecting simulation
Monitor[($cell[42])(alert<>)]
  alert<$cell[42]$threshold(>
|
  $cell[42]$threshold() :
    0:0 1:0 2:0 3:0 4:0
    5:1 6:1 7:1
]

// actuator: writes into cell, affecting the simulation
Inject[($signal)(cell[44]<>)]
  cell[44]<$signal>
]

// router: reads one cell, writes to an adjacent one
Router[($cell[12])(cell[13]<>)]
  cell[13]<$cell[12]$route(>
|
  $cell[12]$route() : ...
]
```

Cell references `$cell[n]` and `cell[n]<>` are resolved at compile time by looking up vertex n in the mesh metadata and generating VHDL port connections to the corresponding placed theng. The compiler’s *coupling table* maintains the mapping from cell indices to fabric positions to VHDL signals.

5 Physical Simulation on the Geometry-Aware Fabric

5.1 Lattice Boltzmann Fluid Dynamics

The lattice Boltzmann method [6] models fluid dynamics by evolving particle distribution functions according to local collision and streaming rules. Each cell computes net forces from its neighbors and updates velocity vectors. In MatterScript IL each fluid cell is a definition whose destinations are its neighbors’ states and whose source is its own next state:

Listing 7: Lattice Boltzmann cell in IL

```
FluidCell[( $n$   $s$   $e$   $w$   $self$ )(next<>)
  next< $n$  $s$  $e$  $w$  $self$ ()>
|
   $n$  $s$  $e$  $w$  $self$ () : // D2Q5 collision and streaming table
  ...
]
```

When this definition is placed geometry-awaredly on a mesh derived from actual riverbed LIDAR data, the neighbor relationships are determined by mesh adjacency rather than a regular grid. The synthetic delays on connections between cells encode the geodesic distances between the physical locations they represent. A narrow channel in the mesh produces faster effective flow not through explicit programming but because cells in the channel have shorter connection delays and therefore propagate state changes more rapidly.

5.2 The Digital Wind Tunnel

The complete system functions as a digital wind tunnel:

1. The physical domain geometry is acquired from field measurement
2. MeshLab converts it to an annotated mesh
3. MatterScript places the simulation logic onto the FPGA fabric conformally, with delay synthesis preserving geometric distances
4. The FPGA runs the simulation with propagation behavior faithful to the physical domain
5. Sensor logic placed at specific cell indices reads simulation state at locations corresponding to physical measurement points

6. Actuator logic writes boundary conditions or forcing terms at specific cells, corresponding to physical inflow or control surfaces

The result is a simulation that can be directly validated against field measurements at known locations, and whose internal dynamics propagate at geometrically correct relative speeds — properties that conventional regular-grid FPGA simulations do not provide.

5.3 Comparison with Conventional FPGA Simulation

Property	Conventional	MatterScript
Domain geometry	Regular grid	Arbitrary mesh
Boundary handling	Staircase approximation	Exact mesh boundary
Cell adjacency	Fixed 4/6-connected grid	Mesh edge adjacency
Propagation delay	Clock-constrained	Geometry-proportional
Timing model	Global clock	Completeness-driven
Placement objective	Minimum wire length	Geometric fidelity
Domain source	Manual grid definition	LIDAR / GeoTIFF / CA
Sensor placement	Post-processing	Embedded cell references

6 The Compiler Pipeline

6.1 Overview

The MatterScript compiler is implemented in Zig 0.17, using the language’s structured IO model for code generation. The current pipeline for the IL dialect is:

$$\text{.ms.il} \xrightarrow{\text{parser}} \text{Network AST} \xrightarrow{\text{emitter}} \text{.vhd} \xrightarrow{\text{GHDL}} \text{.sv} \xrightarrow{\text{Verilator}} \text{native binary}$$

The geometry-aware placement extension adds a preprocessing step:

$$\text{mesh} + \text{metadata} \xrightarrow{\text{placement}} \text{annotated Network AST} \xrightarrow{\text{emitter}} \text{.vhd with delay elements} \xrightarrow{\text{synthesis}} \text{FPGA bitstream}$$

6.2 The IL Parser

The parser is a recursive descent parser on a flat token stream. Whitespace and comments are insignificant. The grammar is:

```

network      = definition* entry?
definition   = name '[' dest-list source-list resolution '|' constants ']'
dest-list    = '(' ('$' name)* ')'
source-list  = '(' (name '<>')* ')'
resolution   = (source-fill | invocation)*

```

```

source-fill = name '<' expr '>'
expr       = ('$' name)+ '()' ?
invocation = name '(' '(' arg* ')' source-list ')'
constants  = (composition ':' (key ':' value)+)*

```

The AST is a flat `Network` containing a slice of `Definition` structs and an optional `EntryInvocation`. No nesting is permitted at the AST level, reflecting the graph rather than tree structure of IL networks.

6.3 VHDL Generation

Each IL definition generates one VHDL entity and architecture. The mapping is:

IL concept	VHDL construct
Destination place	<code>in std_logic_vector(7 downto 0) port</code>
Source place	<code>out std_logic_vector(7 downto 0) port</code>
Completeness	Concurrent AND of all destination valid bits
Name composition	Signal concatenation of 7-bit data fields
Constant table	Combinational <code>case</code> statement
Source fill	Concurrent signal assignment
Synthetic delay	Shift register on NCL signal

6.4 Build System

The build system is implemented in Zig’s build DSL with separate steps:

- `zig build verify` — generates VHDL from all example sources and runs GHDL syntax validation, cross-platform
- `zig build simulate` — full GHDL elaboration, synthesis to Verilog, Verilator compilation, and execution of generated testbenches; Linux only, requires GHDL LLVM backend
- `zig build test` — Zig unit tests for parser and emitter modules

Testbenches are generated automatically from the same IL source as the VHDL, walking the network’s happy path and verifying completeness semantics at each step. No hand-written testbench files are required.

7 Implementation Status

Implemented and functional:

- FSM dialect: parser, VHDL emitter, testbench generator, full GHDL + Verilator simulation pipeline with generated testbenches

- IL dialect: parser, VHDL emitter (combinational), GHDL syntax validation
- Geo dialect: Rule 30 CA engine, mesh generator, OBJ/STL/PGM export, voxel representation
- MKRAND: deterministic random bit generator based on Rule 30, used as a synchronization primitive across dialects
- Build system: cross-platform with verify and simulate steps

Designed, not yet implemented:

- IL component instantiation (one definition invoking another)
- Cell reference syntax and coupling table
- Mesh-to-fabric conformal projection engine
- Delay synthesis from distortion metrics
- FPGA synthesis backend (Vivado/Quartus integration)
- Physical simulation overlay dialect (.ms.ca)
- IL testbench generator

8 Related Work

Asynchronous logic and NCL: Fant [1] and Sparsø [7] provide the theoretical foundations. MatterScript’s contribution is a high-level language that compiles directly to NCL-style VHDL and makes delay synthesis a first-class design operation.

FPGA-based physical simulation: Several groups have implemented lattice Boltzmann and other PDE solvers on FPGAs [11]. These use regular grid placements without geometric awareness. MatterScript introduces geometry-faithful placement as a design objective.

Cellular automata computing: Wolfram [2] and Toffoli and Margolus [8] demonstrated CA as a computational model. MatterScript uses CA as a geometry generator rather than the computational medium.

High-level synthesis: Chisel [10] and SpinalHDL target conventional synchronous RTL. MatterScript targets asynchronous, geometry-aware computation on the same commodity hardware.

Physical computing: Reaction-diffusion [9] and Physarum computing use physical processes as substrates. MatterScript achieves similar coupling between physical geometry and computation in reconfigurable digital hardware, without requiring exotic physical media.

9 Conclusion

MatterScript demonstrates that geometry-aware placement of asynchronous logic networks on commodity FPGA hardware is both theoretically coherent and practically implementable with a small, focused toolchain. The combination of Fant’s Invocation Language, conformal projection of physical domain geometry, and NCL delay synthesis produces simulations whose propagation behavior is faithful to the physical domain without requiring custom hardware.

The working compiler pipeline — from `.ms.il` source through GHDL and Verilator to cycle-accurate simulation — grounds the approach in hardware-verifiable reality. The path to full FPGA deployment is a direct extension of the existing pipeline through standard synthesis tools.

The broader significance is that MatterScript proposes a new optimization objective for FPGA-based simulation: not minimum wire length and maximum clock frequency, but geometric fidelity to the physical domain. For simulation workloads where the geometry of the domain is the problem, this is the right objective. NCL is the enabling technology that makes it practical.

References

- [1] K. M. Fant, *Computer Science Reconsidered: The Invocation Model of Process Expression*. Wiley-Interscience, 2007.
- [2] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002.
- [3] P. Cignoni et al., “MeshLab: an Open-Source Mesh Processing Tool,” in *Proc. Eurographics Italian Chapter Conference*, 2008.
- [4] T. Gingold, *GHDL: Open Source VHDL Simulator*. <https://github.com/ghdl/ghdl>, 2025.
- [5] W. Snyder, *Verilator: The Fastest Verilog/SystemVerilog Simulator*. <https://www.veripool.org/verilator>, 2025.
- [6] S. Succi, *The Lattice Boltzmann Equation: For Fluid Dynamics and Beyond*. Oxford University Press, 2001.
- [7] J. Sparsø and S. Furber, *Principles of Asynchronous Circuit Design*. Springer, 2001.
- [8] T. Toffoli and N. Margolus, *Cellular Automata Machines*. MIT Press, 1987.
- [9] A. Adamatzky, *Physarum Machines: Computers from Slime Mould*. World Scientific, 2010.
- [10] J. Bachrach et al., “Chisel: Constructing Hardware in a Scala Embedded Language,” in *Proc. DAC*, 2012.
- [11] H. M. Waidyasooriya and M. Hariyama, “FPGA-Based Lattice Boltzmann Fluid Simulation,” *International Journal of Parallel Programming*, 2017.